DM546: Compiler Construction

Mandatory Project 1 Due: 2026-11-01

This is one of two **required** projects. Read the following instructions carefully:

- Passing this project is a prerequisite to passing the course.
- It is strongly recommended that you work on this project in groups of **two students** and make a group submission on itslearning. Clearly state both names on the report. If you work alone, you still have to complete all tasks. Submissions with more than two people are not possible.
- Please submit the following:
  - a short report as a PDF without title page (2 to 4 pages; not more!)
  - the source code. Hand in the complete git history, especially the original, unaltered commits on which you base your work.[1] You should use `git bundle create ../project01.bundle --all` to export your repo as a single file.
- Start by cloning the repository at https://gitlab.sdu.dk/dm546/odenc/-/tree/project01. After cloning, make sure to select the `project01` branch, e.g. using the command `git checkout project01`. As a plausibility check, verify the existence of the directory `odenc/langs/lang_num`. If this directory does not exist, you are almost certainly on the wrong branch!
- **AI usage**: Reasonable AI usage is permitted, but it may not interfere with your ability to demonstrate mastery of the learning goals.[2] **The usage of AI has to be declared** by
  - filling in **SDU's GenAI declaration form, and**
  - providing **a full transcript of all prompts**. Omitting prompts is treated as undeclared AI usage.

  Any undeclared AI usage constitutes an exam irregularity ("cheating") and will be reported as such.

[1] This is so I can review diffs relative to the original question posed.

[2] As a rule of thumb, consider what is left if you mark all AI-generated parts as verbatim quotations from a book.

Mandatory Task 1: Numeric literals

Currently `OdenC` only supports numeric literals written as decimal digits. Introduce support for:

- Binary literals with `0b` prefix (e.g., `0b0010`, `-0b01`)
- Decimal literals with no prefix (e.g., `123`, `-12`)
- Hexadecimal literals with `0x` prefix (e.g., `0xa1A`, `-0x1cD`)

The prefixes `0b` and `0x` are case-sensitive (`0B` and `0X` are forbidden), the digits are not.

For better readability of long numbers, the user should be able to group together digits using the underscore `_`, for instance `0x1234_4567` or `1_000_000`. We use the following convention: a number can contain zero or one underscore between each adjacent pair of digits. In other words, a number may **not** start or end with an underscore, nor may it have two adjacent underscores. Underscores within a number have no formal meaning and need to be discarded.

Hint: This task asks only to modify the parser; the interpreter requires no changes.

a) Run `./check.sh` and make sure that no errors are reported. Implement at least 5 test programs and place them in `progs/num/`. The programs need to include a `// TEST` line describing the expected input and output. Running `./check.sh` again should report errors on those programs.

b) Implement support for the generalized numeric literals described above in *LangNum*. You can check your progress by running `./check.sh`.

Mandatory Task 2: Semantic analysis

The goal of this task is to implement a simple "semantic analysis" phase, namely to verify that all variables used in the code have been declared and initialized. Observe that the former is already done by the interpreter at runtime (it will throw an exception if one tries to access a variable that is not declared). In contrast, the new phase you are implementing does this statically *before* the program gets executed.

a) Create the file `odenc/langs/lang_var/semcheck.py` to implement the new class `SemCheck`. Implement the method `def cvd_program(self, prog : Program)` (`cvd` means "check variables [are] declared"). The method does not return any value, but raises the exception `SemCheckUndeclaredVariable` (you have to define it) if an undeclared variable is encountered.

b) Implement a second check `def rbw_program(self, prog : Program)` (`rbw` means "read before write") that raises the exception `SemCheckUninitializedVariable` if a variable would be read before it has been written to.

Remark: We can implement this check easily, since `LangVar` is a straight-line program (with no branching). In general, however, this is not always possible. This is the reason why **in lecture** we **instead** define that any declared variable is initialized with the value `0`.

Thus in this task the program `var x; print(x);` is illegal as `x` is not unitialized, while in lecture we will treat the program equivalent to `var x; x=0; print(x);`.

Mandatory Task 3: Partial evaluation in the front-end

The goal of this task is to optimize *LangVar* ASTs by evaluating fully specified expressions directly in the front-end. For instance the *LangVar* program `print(1+2*3)` should be transformed into `print(7)`, while there is little we can do about `print(input() + input())`.

More formally, we say that a `BinaryOp` node (addition, subtraction, multiplication, division, or modulo) is the root of a "constant expression subtree" if all leaves are of type `Constant` and all inner nodes are of type `BinaryOp`. Your task is to optimize the AST to an equivalent program that does not contain any constant expression subtrees.

a) Implement a function that consumes a `LangVar` AST and returns the total number of `BinaryOp` nodes in the tree.

b) Create the file `odenc/langs/lang_var/partial_eval.py` and implement the class `PartialEval` with the method `def pe_program(self, prog : Program) -> Program`. While you are free to add to the class, make sure[3] the following code returns the optimized program: `PartialEval().pe_program(ast)`.

c) Implement at least one unit test to assert that the optimized programs are equivalent to the original programs. For instance, execute the interpreter on the modified ASTs and check that their behaviour still matches what is described in the `// TEST` line.

Hint: Observe that the existing unit tests are generic and test *all* languages. This may complicate things, since `PartialEval` is only available for *LangVar*. Thus, the easiest approach is likely to implement a new unit test instead of modifying the existing ones — but you are free to chose what works best for you.

d) Report: compare the total number of arithmetic operations before and after the partial evaluation stage. Include all test programs in `progs` and add five more programs to exemplify cases where the optimization performs well and poorly.

Mandatory Task 4: Unary operations

Many programming languages support unary operations; for example, in C we have `~x`, `-x`, and `!x`.[4] The goal of this task is to implement `LangUnary`, which extends `LangVar`.

a) Report: Design a grammar based on `LangVar` which supports unary minus for constants and variables. Make sure that chaining unary minus is **not** permissible; for example, `print(--1)` is forbidden, but `print(1 - -1)` is allowed. Use the same operator precedence (relative to the existing operators) as used in the C language.

Code: Implement the new grammar in `odenc.langs.lang_unary.parser.GRAMMAR` by concatenating it with `odenc.langs.lang_var.parser.GRAMMAR`; that is, specify only the changes (similar to how *LangVar* is defined relative to *LangInt*).

b) Add new AST node type(s) and emit them in `odenc.langs.lang_unary.parser.Transformer`.

c) Add support for the new language constructs in the interpreter. Design a few test programs that include a `// TEST` line and place them in `progs/unary/`. Make sure that your interpreter behaves as expected.

---

[3] preferably by implementing a unit-test ;)

[4] Did you know that `int foo(int& x, int& y) {return x-- - - - - --y;}` is valid C?